

ICP-Flow

The math behind LiDAR scene-flow estimation

A high-level conceptual guide to the mathematics needed to understand the main pipeline. This guide explains the problem, geometry, motion assumptions, association, initialization, and evaluation. It intentionally does not walk through source code or teach ICP from scratch.

The central idea

A LiDAR scan is a set of sampled 3D surfaces. Given scans captured at different times, scene flow assigns a displacement vector to each point, describing where that point moved in a common world reference. ICP-Flow exploits the fact that many points belong to objects that move approximately as rigid bodies: identify corresponding object-like segments, estimate one rigid motion per segment, then apply that motion to its points.

Pipeline at a glance

Stage	Mathematical question
Coordinate alignment	How do we express scans in a comparable frame?
Segmentation	Which nearby points may share one motion?
Association	Which segment in one scan corresponds to which in another?
Rigid registration	What rotation and translation align a matched pair?
Dense flow	What displacement does that transform imply for every point?
Evaluation	How close are predicted displacements to reference flow?

Useful mental model: the difficult part is not only finding a transform. It is deciding which points belong together, which objects correspond across time, and which coordinate frame each transform uses.

1. Scene flow and point-cloud data

Represent a point in scan t as $p(t) = [x, y, z]^T$. Scene flow from time t to time 0 is the 3D displacement that takes that sample to its corresponding location:

$$f(t \rightarrow 0) = p(0) - p(t)$$

The result is a vector per source point, not a new point cloud and not an object label. Its magnitude is distance moved; its direction gives motion direction. The correspondence is conceptual: a LiDAR sensor usually does not observe the exact same surface sample twice, so the algorithm estimates motion from the surrounding shape.

Rigid-body approximation

For a rigid object, all its points share a single motion transform. In homogeneous coordinates, a point is augmented with a 1 and transformed by a 4×4 matrix:

$$\begin{bmatrix} p' \\ 1 \end{bmatrix} = T \begin{bmatrix} p \\ 1 \end{bmatrix}, \quad T = \begin{bmatrix} R & t \\ 0 & 1 \end{bmatrix}$$

Here R is a 3×3 rotation matrix ($R^T R = I$ and $\det(R) = +1$), and t is a 3D translation. A rigid transform preserves distances and angles within the object. The point flow induced by it is:

$$f(p) = (R - I)p + t$$

For pure translation, $R = I$ and every point gets the same vector t . With rotation, flow varies with position: points farther from the rotation center generally have larger displacements. This is why rigid object motion creates a structured, spatially varying scene-flow field.

Assumptions and limits

- A segment is treated as one rigid object over the time interval. Articulated motion, deformable objects, and mixed clusters violate this model.
- A segment may be only part of an object because of occlusion or range limits; different visible parts can still share the same transform.
- Unclustered points cannot reliably borrow an object transform, so practical systems often leave them at zero residual motion or use a fallback.

2. Coordinate frames and ego-motion

A point has coordinates relative to a frame. A LiDAR scan is measured in the sensor/vehicle frame, which moves as the car drives. Comparing raw scans directly mixes two motions: motion of the vehicle and motion of objects in the world. Ego-motion compensation removes the sensor-motion component so stationary scenery approximately lines up.

Transform composition

Let $T_{A \leftarrow B}$ map coordinates expressed in frame B into frame A. The notation makes the direction explicit. To map a point through two frames, compose transforms in application order from right to left:

$$p_A = T_{A \leftarrow B} p_B; \quad p_A = T_{A \leftarrow B} T_{B \leftarrow C} p_C$$

Matrix multiplication is not commutative: generally $T_1 T_2 \neq T_2 T_1$. The order matters. A transform and its inverse have opposite directions: $T_{B \leftarrow A} = (T_{A \leftarrow B})^{-1}$. For rigid transforms, the inverse is $[R^T, -R^T t; 0, 1]$.

Static-world check

Suppose a stationary landmark is seen at p_t in the moving sensor frame and p_0 in the reference sensor frame. A correct ego transform should make $T_{0 \leftarrow t} p_t$ approximately equal p_0 . Residual mismatch can come from pose error, measurement noise, occlusion, or a non-static point.

Separate vehicle and object motion

A useful decomposition is: first map a source point into the reference sensor frame using ego motion; then account for motion of its object relative to that aligned background. If T_{obj} is the estimated residual object transform in the aligned frame and T_{ego} maps source frame to reference frame, total mapping is composed in the correct frame order. The implementation must keep this convention consistent.

A frequent conceptual bug is adding translations from transforms expressed in different frames, or applying the pose on the wrong side of an object transform. Matrix composition is the safe way to combine them.

3. Segmentation and data association

Why group points?

Estimating a free 3D displacement independently for every point is underconstrained when a scan is sparse or surfaces are ambiguous. Grouping points into spatially coherent segments reduces the problem: estimate a transform for a segment, then share it across its points. Ground removal and spatial clustering are practical priors that make this tractable.

Clustering is not object recognition

A density cluster says points are near one another under a chosen distance scale. It does not prove they form one physical object or have one motion. A cluster can merge adjacent objects; one object can split into multiple clusters; sparse regions may be labelled noise. Cluster labels are temporary IDs within a scan, not persistent identities across time.

Matching segments across time

For each source segment i and destination segment j , a matching method needs evidence that the pair can describe the same object. Evidence may include geometric alignment error, overlap, bounding-box compatibility, plausible translation, and plausible rotation. The result is a set of candidate correspondences (i, j) , often with unmatched segments.

Association and registration depend on each other

A bad segment pairing makes registration fail even if the rigid-motion solver is perfect. Conversely, a good initial motion estimate makes geometric matching more reliable. ICP-Flow therefore couples the stages: use coarse motion cues to propose alignments, refine a candidate alignment, and reject implausible matches with consistency checks. This is a form of alternating inference, not a single closed-form solution.

Simple association score (conceptual)

One can imagine a score $C(i,j)$ that combines terms:

$$C(i,j) = w_g E_{\text{geometry}} + w_b E_{\text{box}} + w_m E_{\text{motion}} + w_r E_{\text{rotation}}$$

The weights w_* balance evidence in different units. Lower score may mean better match, depending on convention. This expression is a mental model: actual implementations may use thresholds or a different score.

4. Translation histograms and ICP refinement

You already know ICP generally, so the key extra concept here is its initialization problem. ICP is local: if the starting translation is far from the correct alignment, nearest-neighbour correspondences can be wrong and the iterative fit can settle into a poor local minimum.

Translation as a voting problem

For a source point p_i and a plausible destination point q_j , the translation that would align the pair is $d_{ij} = q_j - p_i$. If many cross-cloud point pairs are considered, true geometric correspondences tend to produce a concentration of similar translation vectors; unrelated pairs tend to spread across translation space.

$H(d)$ = number of point-pair offsets d_{ij} falling in the bin around d

The histogram turns a continuous search over 3D translation into a coarse discrete vote. A strong local peak suggests a likely displacement. Non-maximum suppression keeps spatially separated peaks rather than many neighbouring bins from one peak. Candidate offsets can then be checked against a geometric alignment error, often using nearest-neighbour distances. The selected translation initializes rigid ICP.

Why it helps

- It uses repeated structure across many points rather than trusting one nearest-neighbour assignment.
- It gives a coarse global search over a bounded translation range before local refinement.
- It is naturally robust to many mismatched point pairs when correct offsets create a clear vote concentration.

What it does not solve

A translation histogram does not directly estimate rotation, guarantee a unique peak, or overcome severe shape symmetry. Bin width trades precision for robustness: wide bins merge nearby possibilities; narrow bins fragment noisy votes and require more computation. The search bounds also encode a motion prior. If the true translation lies outside those bounds, the correct initialization cannot be selected.

ICP as local refinement

Starting from a candidate transform, ICP alternates between assigning nearby points and fitting a transform that reduces alignment error. Abstractly, it seeks a transform minimizing a sum of point-to-point or point-to-surface residuals. In this pipeline, ICP refines each proposed segment alignment; validation compares the refined result to the initialization and can reject a refinement that worsens the error.

A low registration error is necessary evidence, not proof of identity: repeated or symmetric shapes can align well under an incorrect pairing.

5. From transforms to dense scene flow

Once a source point has a segment label, the segment association tells us whether a transform is available. If its matched rigid transform is T_{obj} and the ego transform is T_{ego} , compose them to map the point into the target reference frame. Then subtract the original source position:

```
p_target = T_total [p_source ; 1], where T_total is the correctly ordered composition  
f_source = p_target[0:3] - p_source[0:3]
```

This subtraction is the crucial conversion from motion geometry to scene flow. It preserves rotation-induced variation point by point. If a segment is unmatched, the system needs a fallback policy; using identity means zero additional object motion after ego compensation.

A small 2D example

Imagine a point at $p = (1, 0)$ on a body rotating 90 degrees around the origin, with no translation. It moves to $p' = (0, 1)$, so its flow is $(-1, 1)$. A second point at $(2, 0)$ gets flow $(-2, 2)$. The body has one shared rotation, but its points do not share one common flow vector.

Temporal reference matters

The flow direction can be source-to-target or target-to-source. A label such as `flow_0_1` should be read together with dataset conventions, not assumed from its name. For a displacement defined from source t to target 0 , reversing direction changes both sign and correspondence. Evaluation must use the same convention as the ground truth.

Temporal gaps

For scans farther apart in time, displacement magnitude and ambiguity grow. The rigid assumption may remain useful, but objects can turn, disappear, become occluded, or split across clusters. Translation search ranges and plausibility thresholds therefore often depend on the time gap.

6. Evaluation math and reading results

Given predicted flow $\hat{f}_i$ and reference flow f_i for N evaluated points, the endpoint error (EPE) is the Euclidean distance between vectors:

$$\text{EPE}_i = \|\hat{f}_i - f_i\|_2; \text{mean EPE} = (1/N) \sum_i \text{EPE}_i$$

EPE has units of meters and measures absolute vector error. It does not normalize by motion size, so the same error can be relatively more important for a slow point than a fast one.

Threshold metrics

Common scene-flow metrics count a point as accurate when its error is below an absolute tolerance and/or a relative tolerance. Strict and relaxed accuracy differ in how permissive those thresholds are; outlier rate counts points whose error is sufficiently large. Exact thresholds are benchmark-specific, so interpret reported metric names with the evaluation definition used by this repository.

Aggregate carefully

A point-weighted mean gives dense frames or large objects more influence. A scene-weighted mean first averages within each scene and then gives scenes equal influence. Per-frame metrics can expose errors that an overall average hides. Static/dynamic and foreground/background breakdowns answer different questions and depend on the dataset labels.

Sanity checks that connect math to results

- With identical source and target geometry and identity transforms, predicted flow should be near zero.
- For a pure translation, all points in a rigid segment should receive approximately the same flow vector.
- For a rotation, flow should vary smoothly with point position while distances within the transformed segment remain preserved.
- If stationary background flow is large, inspect ego-motion direction, transform order, and flow direction convention.
- A plausible-looking motion with poor endpoint error may indicate a coordinate-frame or ground-truth convention mismatch.

What to learn next in the repository

After this math, a useful reading order is: data conventions and coordinate transforms; segment labels and cluster matching; translation voting; rigid registration; transform composition into flow; metric aggregation. For each stage, track four things: array shape, units, coordinate frame, and source/target time direction. Those four annotations prevent many otherwise subtle errors.

Concept boundary

This guide describes the math at a conceptual level. It does not assert that every candidate score or metric detail above is implemented exactly as written. The repository and dataset conventions determine those specifics.